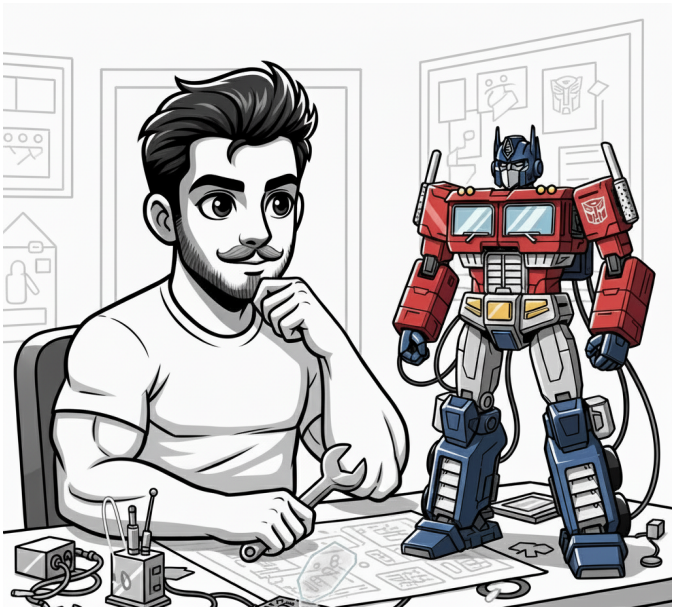
No Libraries, No Shortcuts: LLM from Scratch with PyTorch



OpenAI has recently launched its highly anticipated open-source GPT-OSS models, a moment that invites a minute of reflection on just how far we've come. Years back, even before ChatGPT, I remember reading an article on a GPT model, probably GPT-2, that writes its own essays and poems, and they were just experiments. Fast forward to today, it has already become an integral part of my daily life. And it all started with the landmark "Attention is All You Need" publication in 2017 by Google Research. The Transformer architecture was proposed, which soon powered the very first GPT — GPT-1 (Generative Pretrained Transformer) in 2018.

In the eight years since, progress has been nothing short of extraordinary. Modern large language models now boast multimodal capabilities, advanced reasoning skills, and innovative architectural refinements. Yet, at their core, they still rely on the Transformer skeleton. For many developers today, the intricate brilliance of this underlying design often goes unnoticed, thanks to the accessibility of LLMs today through user-friendly frameworks and APIs.

In this article, we will break apart the Transformer architecture and understand it piece by piece. By the end, you'll have built your own LLM from scratch, which writes brand-new Coldplay songs (who knows, you might just start composing lyrics for Coldplay!).

Table Of Contents

Tokenizer

Any text you provide is broken down by the LLM's tokenizer into smaller units called tokens, which can range from a single character to an entire word.

Consider the text: *"Hold my math!"* Depending on the model's design, tokens can be words, subwords, or even characters.

Word-level tokenization:
`["Hold", "my", "math", "!"]`

Subword-level tokenization:
`["Hold", "my", "ma", "th", "!"]`

Character-level tokenization:
`["H", "o", "l", "d", " ", "m", "y", " ", "m", "a", "t", "h", "!"]`

Next Token Predictors

Large language models are, by basic definition, *next-token predictors*. Given the input tokens, the model learns to analyze and predict the probability of what the next token can be.

Input token	Next Token
Hold m	y

They handle only a fixed number of tokens at once, generating one token per step. Long replies you see, come from repeating this process efficiently. This is achieved by sliding the input window forward after each prediction repeatedly and stopping at an `eos` token or a certain length limit.

Input token (Underlined)	Next Token
Hold<space>m	y
Hold<space>my	<space>
Hold<space>my<space>	m
Hold<space>my<space>m	a
...	
Hold<space>my<space>math!	<|endoftext|>

In fact, you might be thinking that for LLM API calls, you write the code in this way.

```
messages = [
    {
        "role": "system",
        "content": "You are a creative storyteller."
    },
    {
        "role": "user",
        "content": "Write a creative story"
    },
]
```

But after the library processes this, the input that goes to the large language model will be different.

```
"""
<|im_start|>system
You are a creative storyteller.
<|im_end|>
<|im_start|>user
Write a creative story
<|im_end|>
<|im_start|>assistant
"""
```

After pretraining (you'll see that in the coming sections), the large language model is fine-tuned on instructions to make it usable for human interactions. This is called instruction tuning. Otherwise, the LLM will remain as a simple text generator. The above format is how input is passed to instruction-tuned models, although the tags like `<|im_start|>user` may differ for some of them.

Attention is All You Need

So how do LLMs decide what to generate? It has the freedom to generate anything random. But how to make it generate output that makes sense? For that, neural networks need to learn and use context from not only the last token but also from other parts of the input sentence.

Consider an example input,
`The cat chased the`

If the model only looked at the **last token** (`"the"`), it might predict almost anything: `"banana"`, `"man"`, `"moon"`, etc.

But if it uses the **full context** (`"The", "cat", "chased", "the"`), it knows `"mouse"` is far more likely than `"moon"` or `"banana"`.

This needs to work not only in completion tasks but also in translations.

English:

```
I eat a red apple.
```

French:

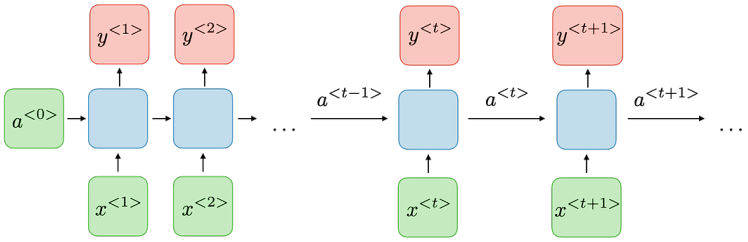
```
Je mange une pomme rouge.
```

The matching words are;

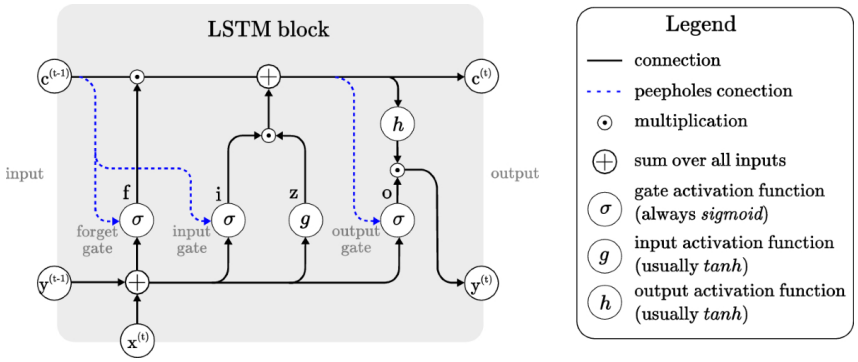
```
English:  [I]     [eat]     [a]      [red]     [apple]
            ↓         ↓        ↓         ↓         ↓
French:   [Je]    [mange]   [une]    [rouge]   [pomme]
```

As you can guess, the word for `red` should come after the `apple` in French. So a word-for-word translation does not work here. Instead, the model should learn and figure out that in such use cases, word order is changed like this.

In the early days of sequence modeling, **Recurrent Neural Networks** **(RNNs)** were the go-to choice for handling text. They processed words one at a time in sequence, passing a "hidden state" forward so each step carried some memory of the past. This worked for short dependencies, but important information from earlier words often faded as the sequence grew longer.



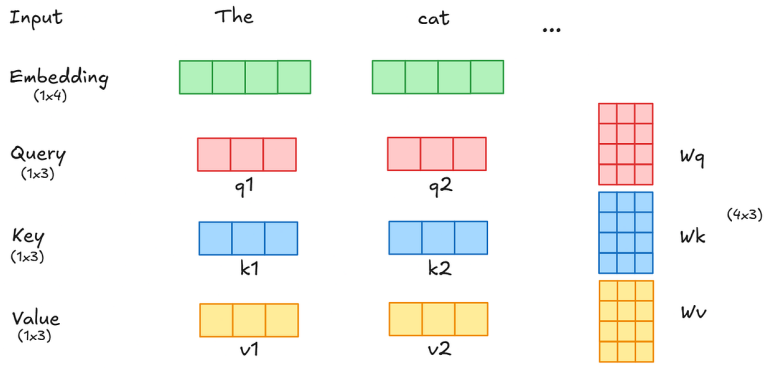
Long Short-Term Memory **(LSTM)** networks, introduced in 1997, improved on this by using special gating mechanisms, namely the input gate, forget gate, and output gate to control the flow of information. These gates decide which parts of the current input and past memory to keep, update, or discard, enabling the network to maintain relevant information over much longer sequences.



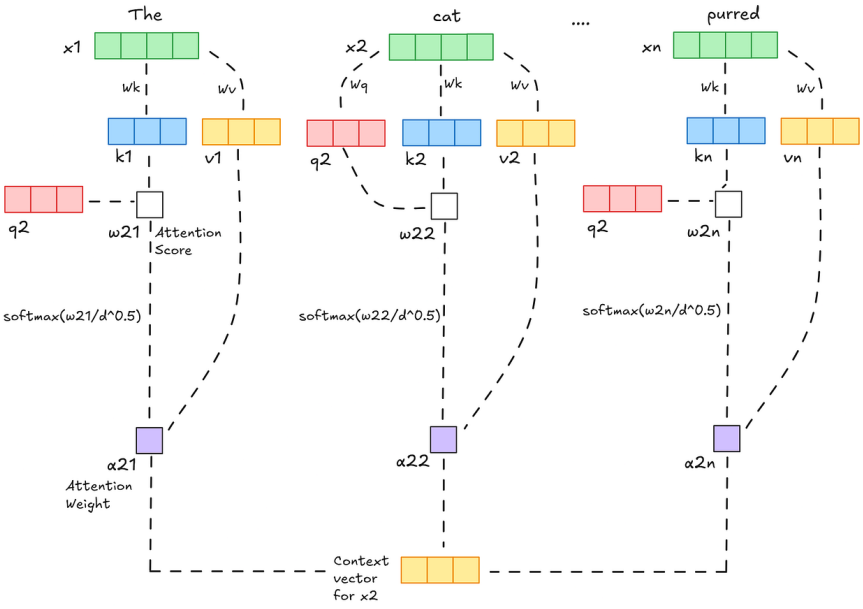
The attention mechanism, introduced in 2017, in [Attention Is All You Need](), addressed this limitation. Instead of relying on sequential processing, attention connects every word in the input directly to every other word, computing weights that determine how much each token should influence the prediction. This allows the model to capture long-range dependencies efficiently and in parallel. Let's do a quick dive into how the attention mechanism works.

The goal is to measure how much each token in a sentence relates to, or influences, every other token. The resulting measure for a pair of tokens is called the **attention score**. Collecting these scores for all token pairs produces the **attention matrix**. For this, we need three components for each token.

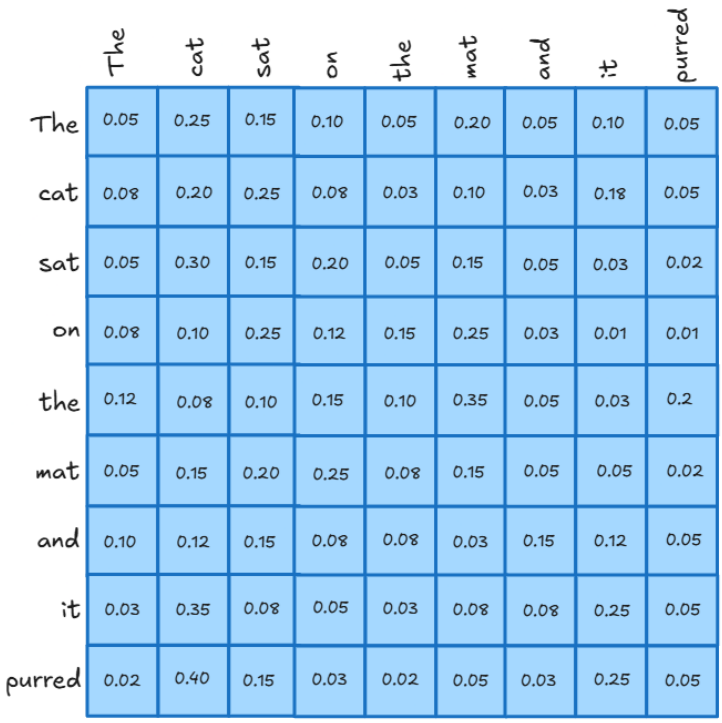
1. **Query vector**
   It represents what this token is looking for in other tokens. It is calculated by the cross product of the input embedding vector and a trainable query matrix `Wq`.
2. **Key vector**
   It represents what this token offers as searchable information. It is calculated by the cross product of the input embedding vector and a trainable key matrix `Wk`.
3. **Value vector**
   It contains the actual information content that will be passed along if the token is attended to. It is calculated by the cross product of the input embedding vector and a trainable value matrix `Wv`.



The score of a token is calculated by taking the dot product of the query vector with the key vector of the respective word we're scoring. For simplicity, let us consider each word in this sentence as a token: "*The cat slept on the mat and it purred.*" So if we're processing the self-attention for the word "slept" at position 3, the first score would be the dot product of q3 and k1. The second score would be the dot product of q3 and k2, and so on. This will give us a score for each token. Each score is normalized using a **softmax activation** and **divided by the square root of the embedding dimension(`d**0.5`)** to get the attention weight. Finally, multiply each value vector by its corresponding attention weight to get the context vector.

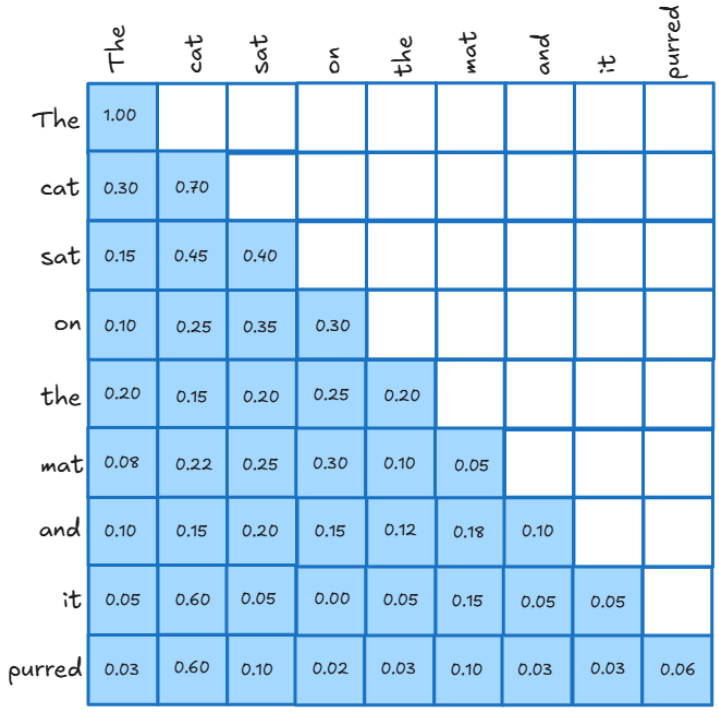


These context vectors are combined as a matrix that we refer to as the attention matrix. In application, all these operations are computed as matrix operations. As you can see, there are higher values for the cells "*it-cat*" and "*purred-cat*", showing higher correlation among these.



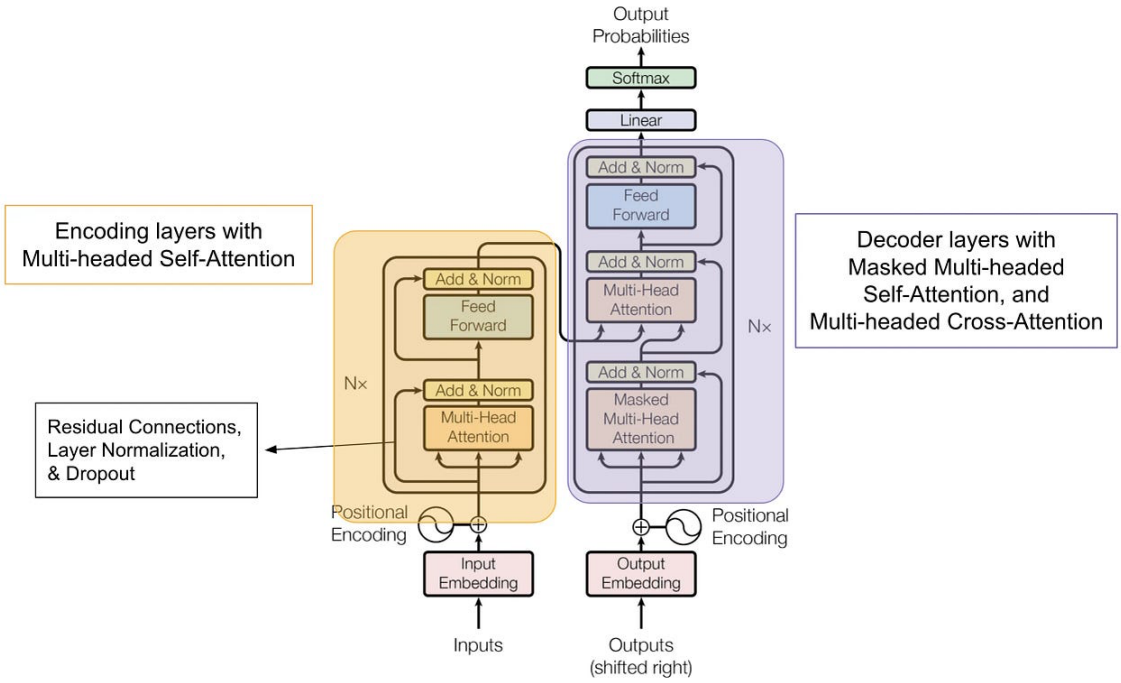
	The	cat	sat	on	the	mat	and	it	purred
The	0.05	0.25	0.15	0.10	0.05	0.20	0.05	0.10	0.05
cat	0.08	0.20	0.25	0.08	0.03	0.10	0.03	0.18	0.05
sat	0.05	0.30	0.15	0.20	0.05	0.15	0.05	0.03	0.02
on	0.08	0.10	0.25	0.12	0.15	0.25	0.03	0.01	0.01
the	0.12	0.08	0.10	0.15	0.10	0.35	0.05	0.03	0.2
mat	0.05	0.15	0.20	0.25	0.08	0.15	0.05	0.05	0.02
and	0.10	0.12	0.15	0.08	0.08	0.03	0.15	0.12	0.05
it	0.03	0.35	0.08	0.05	0.03	0.08	0.08	0.25	0.05
purred	0.02	0.40	0.15	0.03	0.02	0.05	0.03	0.25	0.05

In theory, the probability of a token being chosen as the next one should depend on the past tokens only and not on the future tokens, or else it doesn't make sense. To implement this, we need to mask all the future values in the matrix during the training phase and adjust the row values so that they add up to one as earlier, because these values are probabilities. This is called **Causal Attention,** and you will understand it much better in the upcoming coding part.



	The	cat	sat	on	the	mat	and	it	purred
The	1.00								
cat	0.30	0.70							
sat	0.15	0.45	0.40						
on	0.10	0.25	0.35	0.30					
the	0.20	0.15	0.20	0.25	0.20				
mat	0.08	0.22	0.25	0.30	0.10	0.05			
and	0.10	0.15	0.20	0.15	0.12	0.18	0.10		
it	0.05	0.60	0.05	0.00	0.05	0.15	0.05	0.05	
purred	0.03	0.60	0.10	0.02	0.03	0.10	0.03	0.03	0.06

Building the Transformer Architecture



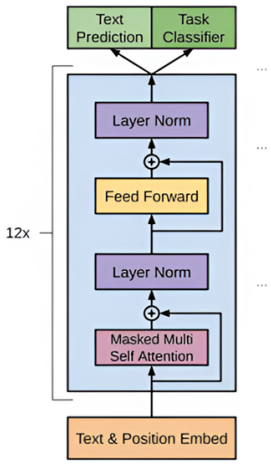
Before diving into code, let's first understand the big picture. A Transformer is built from **encoder and decoder blocks**, each made up of the same key components: **embeddings, positional encodings, self-attention, multi-head attention, and feed-forward layers**.

At its core, think of it as a **modular pipeline**:

1. Input sequence text is broken into tokens.
2. Tokens are converted into numerical embeddings and tagged with positional encodings.
3. Self-attention lets every token "pay attention" to every other token or in simpler words the model figures out how each token relates to others.
4. Multi-head attention combines several self-attention layers and lets the model view those relationships from multiple angles.
5. A feed-forward network transforms the combined information into stronger features.

By stacking many such layers, the model learns increasingly rich representations of language.

Each of the sections below will peel apart one of these building blocks, and we'll implement them step by step in PyTorch until we have a working Transformer from scratch. You will focus on the GPT architecture, which is decoder-only.



Tokenization

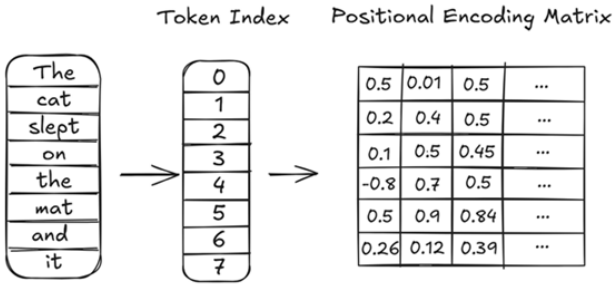
The first step should be to split the input into tokens as we discussed earlier. You can implement character-level tokenization or word-level tokenization from scratch, but here you can use `tiktoken`. Whatever approach you use, the key steps are to tokenize the input text, build a vocabulary that maps each token to an index, and ensure you can convert tokens to numerical IDs for the input, and back from IDs to tokens for the output.

```
import tiktoken

def text_to_token_ids(text, tokenizer, device):
    encoded = tokenizer.encode(text, allowed_special={'<|endoftext|>'})
    encoded_tensor = torch.tensor(encoded).unsqueeze(0).to(device) # add batch dimension and move to device
    return encoded_tensor

def token_ids_to_text(token_ids, tokenizer):
    flat = token_ids.squeeze(0) # remove batch dimension
    return tokenizer.decode(flat.tolist())
```

For custom vocabulary sizes, you can either implement your own byte-pair encoder or simply train a GPT-2 tokenizer using tokenizers library. You can see an implementation in the notebook.

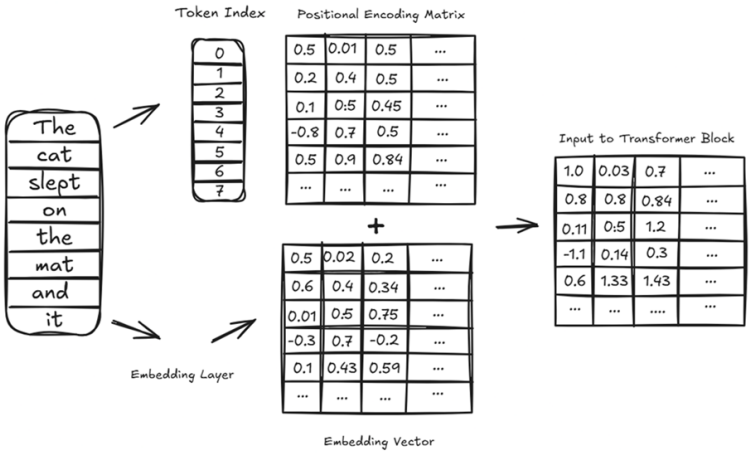
Positional Encoding & Embeddings

Positional encoding encodes the location of each token within a sequence. Transformers rely on it to preserve ordering, since unlike RNNs, they process all tokens in parallel. A simple index isn't expressive enough for long sequences, so positional encoding uses mathematical patterns (often sine and cosine functions) to create a matrix that embeds richer positional information. This gives the model a sense of sequence order while still leveraging parallel computation.



The **embedding layer** is the first step in transforming raw tokens into something a neural network can work with. Each token ID from the vocabulary is mapped to a dense vector of fixed size, known as an **embedding vector**. Instead of representing words as sparse one-hot encodings, embeddings capture semantic meaning, so that related words like *king* and *queen* end up closer together in vector space than unrelated words like *king* and *banana*.

This embedding matrix has dimensions *(vocabulary size × embedding dimension)*, where the embedding dimension is a hyperparameter you choose. The dimensions of the positional encoding matrix should match those of the embedding matrix because, as you can see in the architecture, the embedding matrix is enriched with information by summing up with the positional encoding matrix element-wise.



For simplicity, assume that the embedding dimension and attention dimension are the same for output. For input, the embedding layer represents the whole vocabulary of words. If the model is trained on 10000 unique words (tokens literally), that will be the input shape. For GPT-2, the standard is 50257 words. Similarly, the input for the positional encoding layer will be equal to the number of tokens the input layer of the LLM has (number of tokens the LLM processes at one go). Call it the **context length**.

```
self.embedding = torch.nn.Embedding(vocab_size, attention_dim)
self.positional_embedding = torch.nn.Embedding(context_length, attention_dim)
```

In the forward method, define the flow as.

```
embeddings = self.embedding(context)
context_len = context.shape[1]
position = torch.arange(context_len, device=context.device).unsqueeze(0)
position_embeddings = self.positional_embedding(position)

e = embeddings + position_embeddings
```

You will see it in action in the full GPT decoder that we will build soon.

Self-Attention: How Tokens Gossip About Each Other

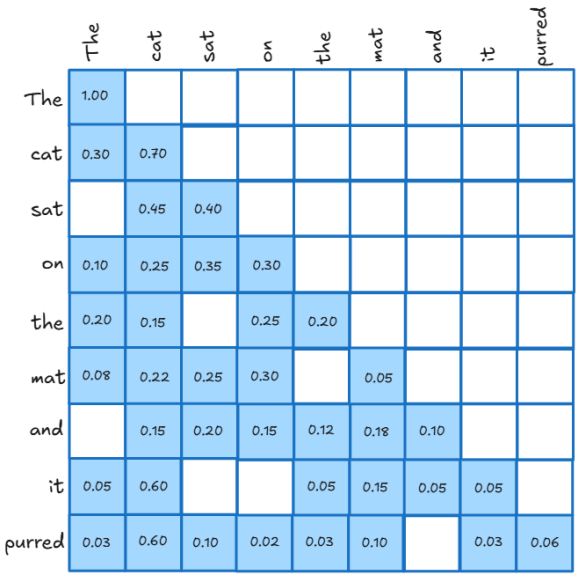
Now, let us code the self-attention module using what we discussed earlier in the section. Define the trainable query, key, and value matrices.

```
self.w_key = torch.nn.Linear(embed_dim, attention_dim, bias=bias)
self.w_query = torch.nn.Linear(embed_dim, attention_dim, bias=bias)
self.w_value = torch.nn.Linear(embed_dim, attention_dim, bias=bias)
```

Compute the query, key, and value vectors.

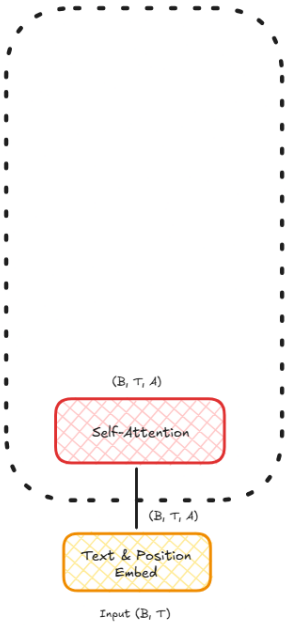
```
k = self.w_key(x)   # (B, T, A)
q = self.w_query(x) # (B, T, A)
v = self.w_value(x) # (B, T, A)

```
where,
B: batch size,
T: context length,
A: attention dimension
```

Now, compute the attention scores using the product of the query and key vectors. Take the transpose of the key vector to match dimensions. Also, before applying softmax, normalize the value by dividing it by the square root of the embedding dimension.

$$\text{Scores} = \frac{QK^T}{\sqrt{d_k}}$$

```
scores = (q @ k.transpose(-2, -1)) / (k.size(-1) ** 0.5)  # (B, T, T)
```

Mask the future positions and apply softmax activation. Finally, return the product of attention weights and value vector.

```
mask = torch.triu(torch.ones(T, T, device=x.device), diagonal=1).bool()
scores = scores.masked_fill(mask, float('-1e10'))

attn = scores.softmax(dim=-1)  # (B, T, T)
final = attn @ v  # (B, T, A)
```

Don't forget to add dropouts as needed to stabilize the training. It masks additional attention weights randomly according to the percentage you specify. Here it is 0.1, meaning 10% of the weights are masked randomly.
Dropout is considered for the entire matrix, not just the remaining values.
```



Putting it all together as a module,

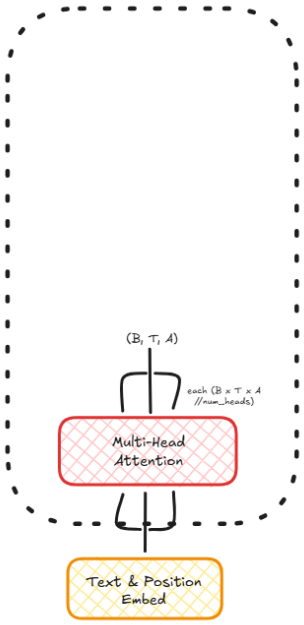
```
class SelfAttention(torch.nn.Module):
    def __init__(self, embed_dim, attention_dim, bias=False, dropout=0.1):
        super().__init__()
        self.w_key = torch.nn.Linear(embed_dim, attention_dim, bias=bias)
        self.w_query = torch.nn.Linear(embed_dim, attention_dim, bias=bias)
        self.w_value = torch.nn.Linear(embed_dim, attention_dim, bias=bias)
        self.dropout = torch.nn.Dropout(dropout)

    def forward(self, x):
        B, T, _ = x.size()

        k = self.w_key(x)   # (B, T, A)
        q = self.w_query(x) # (B, T, A)
        v = self.w_value(x) # (B, T, A)

        # Scaled dot-product attention
        scores = (q @ k.transpose(-2, -1)) / (k.size(-1) ** 0.5)  # (B, T, T)

        # Causal mask (future positions masked)
        mask = torch.triu(torch.ones(T, T, device=x.device), diagonal=1).bool()
        scores = scores.masked_fill(mask, float('-1e10'))

        attn = scores.softmax(dim=-1)  # (B, T, T)

        attn = self.dropout(attn)

        return attn @ v  # (B, T, A)
```



Multi-Head Attention: The Group Chat in Your Model's Brain

You have a self-attention module ready. But for a massive model that is being trained on massive datasets, this won't be enough. Thus, we combine multiple attention heads in parallel to create multi-head attention. The core idea is that multiple attention heads can focus on different aspects of the data, capturing relationships across various subspaces and positions, which allows the model to learn richer and more complex patterns.

```
class MultiHeadAttention(torch.nn.Module):
    def __init__(self, num_heads, embed_dim, attention_dim, dropout=0.1):
        super().__init__()
        self.head_size = attention_dim//num_heads
        self.heads = torch.nn.ModuleList()
        for i in range(num_heads):
            self.heads.append(SelfAttention(embed_dim=embed_dim, attention_dim=self.head_size,dropout=dropout))

    def forward(self,x):
        head_outputs = []
        for head in self.heads:
            head_outputs.append(head(x)) #B x T x A//num_heads
        concatenated = torch.cat(head_outputs, dim = 2)
        return concatenated
```

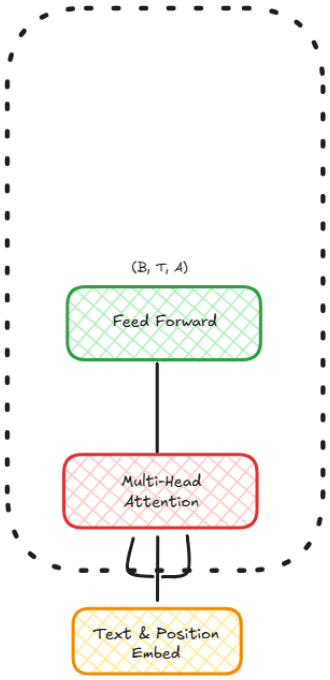


The total attention dimension is divided among the heads, enabling parallel processing. The resulting attention matrix outputs are concatenated to get a full result.

Feed-Forward Networks

After attention, each token embedding is passed through a small feed-forward network. This could simply be two linear layers, which consist of an up-projection and a down-projection with a non-linear activation in between. You can make it more complex according to your architecture.

```
class FeedForward(torch.nn.Module):
    def __init__(self,attention_dim):
        super().__init__()
        self.up = torch.nn.Linear(attention_dim,attention_dim*4)
        self.gelu = torch.nn.GELU()
        self.down = torch.nn.Linear(attention_dim*4,attention_dim)
    def forward(self,x):
        return self.down(self.gelu(self.up(x)))
```



The Decoder with Residual Connections

Now that you have the multi-head attention module and the feed-forward network, let us build the decoder part of the model.

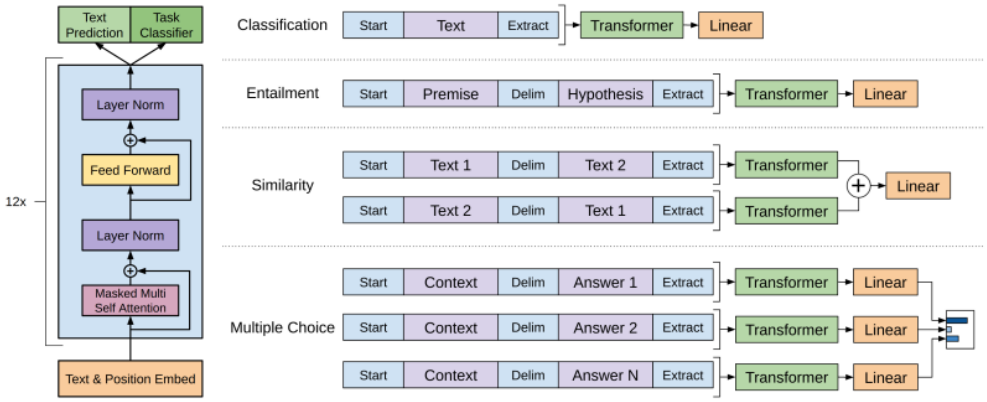


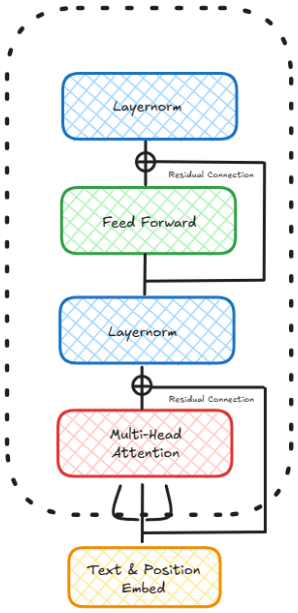
Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

Closely observe the diagram here. You'll notice that each sub-layer (attention or feed-forward) doesn't simply replace its input. Instead, the original input is **added back to the output** of the sub-layer. This shortcut is called a **residual connection**. It helps the model train more effectively by preserving the original signal and avoiding issues like vanishing gradients.

On top of this, every residual connection is followed by **Layer Normalization**. Unlike **BatchNorm**, which normalizes across a batch, **LayerNorm** normalizes across the features of a single token embedding, centering the activations of a neural network layer around a mean of 0 and normalizing their variance to 1. This ensures that the scale and distribution of activations remain stable, which is critical in deep networks like Transformers, where stacking many layers can otherwise destabilize training.

Stack up the layers as in the diagram, and you will have this.

```
class Decoder(torch.nn.Module):
    def __init__(self,num_heads,embed_dim,attention_dim, dropout=0.1):
        super().__init__()
        self.masked_multihead = MultiHeadAttention(num_heads, embed_dim, attention_dim, dropout)
        self.feed_forward = FeedForward(attention_dim)
        self.n1 = torch.nn.LayerNorm(attention_dim)
        self.n2 = torch.nn.LayerNorm(attention_dim)
    def forward(self,x):
        e = self.masked_multihead(self.n1(x))
        e =  e + x
        e = self.feed_forward(self.n2(e))
        return e
```



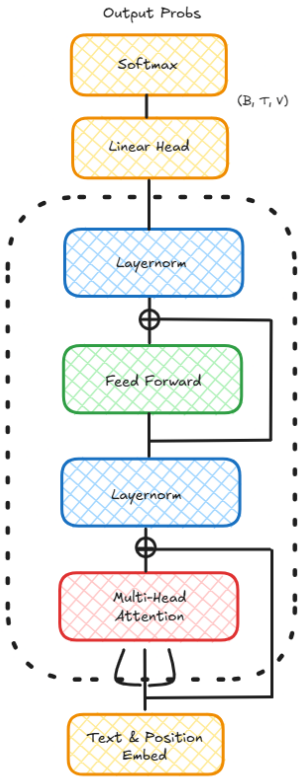
Putting It All Together: The Transformer Skeleton

Perfect. Now, you just need to design the input and output parts of the model. The input part will be the embedding and positional encoding layer, which we created earlier. They convert raw token IDs into dense vectors enriched with positional information, preparing them for the Transformer blocks.

The output part gets a little more interesting. You learned that the LLM output must represent a probability distribution over the entire vocabulary. That means, every token we expect it to learn should have its own output, or in other words, the **output size equals the vocabulary size**. So we project them linearly through a layer of the vocabulary size, which is also called the **LM (language modelling) head**. The LM head is just a linear projection without activation. The output is still not probabilities, but can be called **logits**. ***Softmax*** is applied afterward during training/inference to get the probability distribution.

```
import torch
from torch import nn

class GPT(nn.Module):
    def __init__(self, num_heads, vocab_size, embed_dim, attention_dim, num_blocks, context_length, dropout_rate):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, attention_dim)
        self.positional_embedding = nn.Embedding(context_length, attention_dim)

        self.decoders = nn.ModuleList([
            Decoder(num_heads, attention_dim, attention_dim, dropout_rate) for _ in range(num_blocks)
        ])

        self.exit_norm = nn.LayerNorm(attention_dim)
        self.linear = nn.Linear(attention_dim, vocab_size)

    def forward(self, context):
        embeddings = self.embedding(context)
        context_len = context.shape[1]
        position = torch.arange(context_len, device=context.device).unsqueeze(0)
        position_embeddings = self.positional_embedding(position)

        e = embeddings + position_embeddings

        for decoder in self.decoders:
            e = decoder(e)

        return self.linear(self.exit_norm(e))
```

Multiple decoders are added according to the scale you require. For large GPT models, this can go up to 25 heads, which may seem easy, but will consume a ton of memory and require massive data to train.



So far, so good. You can test your model using a simple sequence generating function.

```
def top_k_logits(logits, k):
    v, ix = torch.topk(logits, k)
    out = logits.clone()
    out[out < v[:, [-1]]] = float('-inf')
    return out

def generate(model, max_new_tokens, context, context_length, temperature=1.0, top_k=None):
    res = []
    for _ in range(max_new_tokens):
        if context.shape[1] > context_length:
            context = context[:, -context_length:]

        logits = model(context)  # [B, T, V]
        logits = logits[:, -1, :]  # [B, V]
        logits = logits / max(temperature, 1e-3)

        if top_k is not None:
            logits = top_k_logits(logits, top_k)

        if torch.isnan(logits).any() or torch.isinf(logits).any():
            raise ValueError("Logits contain NaN or Inf")

        probabilities = nn.functional.softmax(logits, dim=-1)
        probabilities = torch.clamp(probabilities, min=1e-9, max=1.0)

        next_token = torch.multinomial(probabilities, 1)  # [B, 1]
        context = torch.cat((context, next_token), dim=1)


    return context


start_context = "I want something"
model = GPT(num_heads,vocab_size,embed_dim,attention_dim,num_blocks,context_length, dropout_rate).to(device)
model.eval()
token_ids = generate(
    model=model,
    context=text_to_token_ids(start_context, tokenizer, device),
    max_new_tokens=10,
    context_length=context_length
)
print("Output text:\n", token_ids_to_text(token_ids, tokenizer))
```

If things went right, you can see some random garbage output like this.

```
Output text:
 I want something introduceウ coaches Kard Judaism trendsCommerce rotating infiltration approach
```

Model Pretraining

The model pretraining is, in simple words, intended to enable the model to understand and speak basic English. The model must be able to generate a grammatically correct sequence of words that makes some sense, although there may not be much context. For this, we need a dataset that can provide a large corpus of English text. You can choose from numerous publicly available datasets, such as IMDb.

Data Preparation

Convert the given dataset into continuous text data and remove unwanted characters that create noise.

```
from datasets import load_dataset
import re

# Load dataset
ds = load_dataset("stanfordnlp/imdb")

# Keep only English (ASCII) characters
def keep_english_only(text):
    return re.sub(r"[^\x00-\x7F]+", "", text)

# Clean and combine a list of texts
def combine_and_clean(text_list):
    # Keep only English
    cleaned_list = [keep_english_only(t) for t in text_list]
    # Combine into one string
    combined = " ".join(cleaned_list)
    # Remove extra spaces/newlines
    combined = re.sub(r'\s+', ' ', combined).strip()
    return combined

# Create separate combined strings
train_text_data = combine_and_clean(ds['train']['text'])
test_text_data = combine_and_clean(ds['test']['text'])
```

We need data in the format given below for this concept to be implemented.

```
input_ids: [101, 102, 103, 104, 105]

untokenized input_ids: ["The", "cat", "sat", "on", "the"]

target_ids: [102, 103, 104, 105, 106]

untokenized target_ids: ["cat", "sat", "on", "the", "mat"]
```

Thus, define the dataloader that splits our dataset accordingly.

```
from torch.utils.data import Dataset, DataLoader

class CustomDataset(Dataset):
    def __init__(self, txt, tokenizer, max_length, stride):
        self.input_ids = []
        self.target_ids = []

        # Tokenize the entire text
        token_ids = tokenizer.encode(txt, add_special_tokens=False)

        # Use a sliding window to chunk the data into overlapping sequences of max_length
        for i in range(0, len(token_ids) - max_length, stride):
            input_chunk = token_ids[i:i + max_length]
            target_chunk = token_ids[i + 1: i + max_length + 1]
            self.input_ids.append(torch.tensor(input_chunk))
            self.target_ids.append(torch.tensor(target_chunk))

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
        return self.input_ids[idx], self.target_ids[idx]


def create_encoded_dataloader(txt, tokenizer, batch_size=4, max_length=128,
                         stride=128, shuffle=True, drop_last=True, num_workers=0):

    # Create dataset
    dataset = CustomDataset(txt, tokenizer, max_length, stride)

    # Create dataloader
    dataloader = DataLoader(
```

```
        dataset, batch_size=batch_size, shuffle=shuffle, drop_last=drop_last, num_workers=num_workers, pin_memory=True)

    return dataloader

total_characters = len(train_text_data)
total_tokens = len(tokenizer.encode(train_text_data))

print("Characters:", total_characters)
print("Tokens:", total_tokens)

# Sanity check

if total_tokens * (0.95) < context_length:
    print("Not enough tokens for the training loader. "
          "Try to lower the context_length or "
          "increase the `training_ratio`")

if total_tokens * (1-0.95) <context_length:
    print("Not enough tokens for the validation loader. "
          "Try to lower the context_length or "
          "decrease the `training_ratio`")


train_dataloader = create_encoded_dataloader(
    train_text_data,
    tokenizer=tokenizer,
    batch_size=2,
    max_length=context_length,
    stride=context_length,
    shuffle=True,
    drop_last=True
)

test_dataloader = create_encoded_dataloader(
    test_text_data,
    tokenizer=tokenizer,
    batch_size=2,
    max_length=context_length,
    stride=context_length,
    shuffle=False,
    drop_last=True
)
```

Training

Before training, you need to initialize the weights to ensure your model starts training from a predefined point. This is the standard practice for GPT models.

```
def initialize_weights(module):
    if isinstance(module, nn.Linear):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
            torch.nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
    elif isinstance(module, nn.LayerNorm):
        torch.nn.init.ones_(module.weight)
        torch.nn.init.zeros_(module.bias)

model.apply(initialize_weights)
```

Let us start with the loss function. LLMs are, in a way, performing multi-class classification, where each possible word in the vocabulary is a class, and the model outputs a probability distribution over all words. Thus, we use the **cross-entropy loss**.

$$[L = -\sum_{i=1}^{V} y_i \log(p_i), \quad \text{where } V \text{ is the vocabulary size, } y_i = \begin{cases} 1 & \text{if token } i \text{ is the correct next word} \\ 0 & \text{otherwise} \end{cases}, \ p_i \text{ is the predicted probability for token } i.]$$

Cross-entropy rewards high probability for the right word, and punishes when the correct word has low probability.

Consider our previous example.

```
Index: Token
0 → "The"           Inputs: ["The", "cat", "sat", "on", "the"]
1 → "cat"           Targets: ["cat", "sat", "on", "the", "mat"]
2 → "sat"
3 → "on"
4 → "the"
5 → "mat"
```

Position	Input token	Prediction (probabilities over vocab)	Predicted word	Target	Correct?
101	"The"	[0.05, 0.90, 0.02, 0.01, 0.01, 0.01]	"cat"	"cat"	✅
102	"cat"	[0.05, 0.05, 0.10, 0.70, 0.05, 0.05]	"on"	"sat"	❌
103	"sat"	[0.05, 0.05, 0.05, 0.05, 0.70, 0.10]	"the"	"on"	❌
104	"on"	[0.05, 0.05, 0.05, 0.05, 0.75, 0.05]	"the"	"the"	✅
105	"the"	[0.05, 0.05, 0.05, 0.05, 0.05, 0.75]	"mat"	"mat"	✅

Computing loss for each prediction,

- **Position 0:** Target = `"cat"` → P = 0.90 → L0 = $-\log f_0(0.9) \approx 0.105$
- **Position 1:** Target = `"sat"` → P = 0.10 → L1 = $-\log f_0(0.1) = 2.302$
- **Position 2:** Target = `"on"` → P = 0.05 → L2 = $-\log f_0(0.05) = 2.996$
- **Position 3:** Target = `"the"` → P = 0.75 → L3 = $-\log f_0(0.75) \approx 0.288$
- **Position 4:** Target = `"mat"` → P = 0.75 → L4 = $-\log f_0(0.75) \approx 0.288$

Taking the average loss,

$Lavg = (0.105+2.302+2.996+0.288+0.288) / 5 \approx 1.20$

Since wrong predictions give huge loss values, a few bad predictions dominate the average loss. Using this loss, define evaluation functions. We have explicitly mentioned `@torch.no_grad()` because the model weights shouldn't be updated while the loss is being calculated.

```
criterion = nn.CrossEntropyLoss()

def calc_loss_batch(input_batch, target_batch, model, device):
    input_batch = input_batch.to(device, non_blocking=True)
    target_batch = target_batch.to(device, non_blocking=True)

    logits = model(input_batch)  # [B, T, V]
    B, T, V = logits.shape
    loss = criterion(logits.view(B * T, V), target_batch.view(B * T))
    return loss


@torch.no_grad()
def calc_loss_loader(data_loader, model, device, num_batches=None):
    if len(data_loader) == 0:
        return float("nan")

    model.eval()
    total_loss = 0.0
    num_batches = len(data_loader) if num_batches is None else min(num_batches, len(data_loader))

    for i, (inp, tgt) in enumerate(data_loader):
        if i >= num_batches:
            break
        loss = calc_loss_batch(inp, tgt, model, device)
        total_loss += loss.item()

    model.train()
    return total_loss / num_batches

@torch.no_grad()
def evaluate_model(model, train_loader, val_loader, device, eval_iter=1):
    train_loss = calc_loss_loader(train_loader, model, device, num_batches=eval_iter)
    val_loss   = calc_loss_loader(val_loader, model, device, num_batches=eval_iter)
    return train_loss, val_loss
```

Note that you have explicitly set `model.eval()` while calculating loss, set it back to `model.train()` once done.
When it comes to training, the biggest challenge is how to adjust the learning rate over time. If it's too high, the model won't converge. If it's too low, training will be painfully slow. To balance this, we often use a scheduler that adapts the learning rate during training.

In this setup, we use a custom scheduler called `CosineWithWarmup`.

```
class CosineWithWarmup(torch.optim.lr_scheduler._LRScheduler):
    def __init__(self, optimizer, warmup_steps, total_steps, base_lr, min_lr, last_epoch=-1):
```

```
        self.warmup_steps = max(1, warmup_steps)
        self.total_steps = max(self.warmup_steps + 1, total_steps)
        self.base_lr = base_lr
        self.min_lr = min_lr
        super().__init__(optimizer, last_epoch)

    def get_lr(self):
        step = self.last_epoch + 1
        lrs = []
        for _ in self.base_lrs:
            if step <= self.warmup_steps:
                lr = self.base_lr * step / self.warmup_steps
            else:
                progress = (step - self.warmup_steps) / max(1, self.total_steps - self.warmup_steps)
                lr = self.min_lr + 0.5 * (self.base_lr - self.min_lr) * (1 + math.cos(math.pi * progress))
            lrs.append(lr)
        return lrs
```

- **Warmup phase:** For the first few steps, the learning rate linearly ramps up from 0 to the base value. This helps stabilize training, especially for large models like GPT, which can otherwise diverge early on.
- **Cosine decay:** After warmup, the learning rate gradually decreases following a cosine curve, smoothly decaying towards a minimum value (`min_lr`). This prevents sudden drops and helps the model "settle" into a good local minimum.

Start small → Rise steadily → Decay smoothly.

```
settings = {
    "learning_rate": 3e-4,
    "weight_decay": 0.1,            # Standard for GPT-style training
    "num_epochs": 300,
    "batch_size": 32,               # Balance for GPU memory vs convergence
    "warmup_steps": 1500,           # Warmup helps avoid divergence early
    "max_lr": 3e-4,
    "min_lr": 3e-5,
    "eval_freq": 200,
    "eval_iter": 20,
    "gradient_clip": 1.0,
    "patience": 50,
    "min_improvement": 1e-4,
    "print_interval": 1,
    "generate_interval": 5
}


train_dataloader = create_encoded_dataloader(
    train_text_data,
    tokenizer=tokenizer,
    batch_size=settings["batch_size"],
    max_length=context_length,
    stride=context_length,
    shuffle=True,
    drop_last=True
)

test_dataloader = create_encoded_dataloader(
    test_text_data,
    tokenizer=tokenizer,
    batch_size=settings["batch_size"],
    max_length=context_length,
    stride=context_length,
    shuffle=False,
    drop_last=True
)
```

It's a convenient practice to manage your settings in one block. The next piece is the training loop, where the actual learning happens.

```
def train_model(
    model,
    train_loader,
    val_loader,
    device,
    settings,
    save_path="checkpoints/gpt_256_256_8_8.pt",
):

    torch.manual_seed(123)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(123)

    model.to(device)

    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=settings["learning_rate"],
        weight_decay=settings["weight_decay"],
        betas=(0.9, 0.95),
    )

    total_steps = settings["num_epochs"] * len(train_loader)
    scheduler = CosineWithWarmup(
        optimizer,
```

```
        warmup_steps=settings["warmup_steps"],
        total_steps=total_steps,
        base_lr=settings["max_lr"],
        min_lr=settings["min_lr"],
    )

    train_losses, val_losses, tokens_seen_track = [], [], []
    tokens_seen, global_step = 0, -1
    best_val_loss, patience_counter = float("inf"), 0

    for epoch in range(settings["num_epochs"]):
        model.train()
        for step, (inp, tgt) in enumerate(train_loader):
            loss = calc_loss_batch(inp, tgt, model, device)
            loss.backward()

            # gradient clipping
            torch.nn.utils.clip_grad_norm_(model.parameters(), settings["gradient_clip"])

            optimizer.step()
            optimizer.zero_grad(set_to_none=True)
            scheduler.step()
            global_step += 1
            tokens_seen += inp.numel()

            # evaluation
            if global_step % settings["eval_freq"] == 0:
                train_loss, val_loss = evaluate_model(
                    model, train_loader, val_loader, device,
                    eval_iter=settings["eval_iter"],
                )
                train_losses.append(train_loss)
                val_losses.append(val_loss)
                tokens_seen_track.append(tokens_seen)
                lr_now = optimizer.param_groups[0]["lr"]

                print(f"Ep {epoch+1} | step {global_step:06d} | lr {lr_now:.3e} "
                      f"| train {train_loss:.3f} | val {val_loss:.3f}")

                # early stopping
                if val_loss + settings["min_improvement"] < best_val_loss:
                    best_val_loss = val_loss
                    patience_counter = 0
                    os.makedirs(os.path.dirname(save_path) or ".", exist_ok=True)
                    torch.save({
                        "model_state": model.state_dict(),
                        "optimizer_state": optimizer.state_dict(),
                        "epoch": epoch,
                        "global_step": global_step,
                    }, save_path)
                    print(f"[Checkpoint saved at step {global_step}]")
                else:
                    patience_counter += 1
                    if patience_counter >= settings["patience"]:
                        print("Early stopping triggered.")
                        return train_losses, val_losses, tokens_seen_track

    return train_losses, val_losses, tokens_seen_track
```

There are three things worth mentioning in the code. These are small details, but they make a big difference in training stability and efficiency.

1. **Gradient Clipping**
   During backpropagation, the model calculates gradients for each parameter. In practice, sometimes these gradients can become extremely large (**exploding gradients**), especially in deep networks or when training on long sequences. If that happens, the weight updates can destabilize training and cause the loss to diverge. Gradient clipping prevents this by putting a ceiling on the size of the gradients.
2. **Early Stopping**
   Training LLMs takes days. So it's necessary to keep a check on the compute and cut off if there is no meaningful improvement.
3. **AdamW**
   For the actual weight updates, we use `AdamW`, a modern optimizer that combines the benefits of Adam with proper **weight decay regularization**. Adam adapts the learning rate individually for each parameter, which helps models converge faster. The “W” stands for *decoupled weight decay*. Unlike classic Adam, it cleanly separates weight decay from gradient updates, which improves generalization.

This is some random output I got.

```
the movie starts slow and i thought it was going to be boring, but then going to be interesting.
the acting is okay, some are boring felt like they just gave up.
still, it was not the worst film i’ve seen
```

Teaching Your Model to Follow (and Sing Coldplay)

Congratulations! After some long waiting, you have your own LLM capable of speaking basic English (although it may not make much logical sense as we trained on IMDb and the architecture is small and basic). Now, time to teach it Coldplay style patterns. To do this, we fine-tune the model on a small Coldplay dataset. This is actually how large language models like GPT are built:

- First, they undergo **pretraining** on a massive general-purpose dataset (billions of tokens from books, websites). This teaches them grammar, vocabulary, and general world knowledge.
- Then, they are **fine-tuned** on smaller, specialized datasets to adapt them to a particular style or task (chatting, coding, medical Q&A, legal reasoning, etc.). Without fine-tuning, GPT would just be a giant text predictor. With fine-tuning, it becomes conversational, safe, and aligned to the tasks we actually care about.

You can use this dataset and use the same preprocessing steps to get it ready. Use the same training loop for fine-tuning, but tweak the settings a bit.

```
settings_ft = {
    "learning_rate": 1e-5,          # Lower LR for fine-tuning to preserve pretrained weights
    "weight_decay": 0.01,           # Reduced weight decay
    "num_epochs": 5,                # Fewer epochs since Coldplay dataset is small
    "batch_size": 4,                # Smaller batch size for small dataset
    "warmup_steps": 100,            # Shorter warmup
    "max_lr": 1e-5,
    "min_lr": 1e-6,
    "eval_freq": 50,
    "eval_iter": 5,
    "gradient_clip": 0.5,           # gentler clipping
    "patience": 3,
    "min_improvement": 1e-4,
    "print_interval": 1,
    "generate_interval": 2
}

train_losses_ft, val_losses_ft, tokens_seen_ft = train_model(
    model,
    train_dataloader_ft,
    val_dataloader_ft,
    tokenizer,
    device,
    settings=settings_ft,
    context_length=context_length,
    save_path="checkpoints/gpt_512_512_8_8_finetuned_coldplay.pt",
    sample_prompt="Look at the star look how the "
)
```

Let’s check the output once again,

```
lights go out and the stars begin to fall i hear your voice across the night
lights are running in circles chasing the echoes
you are the star that keeps me alive
Oh-ooh-oh-ooh oh, oh
i will follow you, i will follow you
```

Wrapping Up

And that’s a wrap! You have built your own transformer architecture from scratch and trained it with nothing but pure PyTorch. You have seen everything you need from concepts to code, with more intricate information explained that you won’t find together anywhere else. Here is the complete notebook.

Of course, this doesn’t end here. Modern AI has a lot of advanced architectures and algorithms built on top of this to enable the astonishing capabilities we see today. I will definitely be sharing more deep dives in the coming days. It’s fine if you feel these concepts a bit complex to understand in one go. My goal was to distill the entire learning process I went through into one place, so you can focus on *building* rather than searching the web. Drop some claps and follow if you liked it (I am sure you did!).